

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
```

```
!kaggle datasets download mohammadamireshraghi/blood-cell-cancer-all-4class
```

Warning: Your Kaggle API key is readable by other users on this system! To fix this, you can run 'chmod 600 /root/.kaggle/kaggle.json'

Dataset URL: <https://www.kaggle.com/datasets/mohammadamireshraghi/blood-cell-cancer-all-4class>

License(s): Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)

Downloading blood-cell-cancer-all-4class.zip to /content

100% 1.68G/1.68G [00:17<00:00, 106MB/s]

```
import zipfile
zip_ref = zipfile.ZipFile('/content/blood-cell-cancer-all-4class.zip', 'r')
zip_ref.extractall('/content')
zip_ref.close()
```

```
import numpy as np
import pandas as pd
import random as rd
import os
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras import preprocessing
from tensorflow.keras import metrics
#setting seed for reproducibility
from numpy.random import seed
seed(25)
tf.random.set_seed(50)
```

```
import os
# List contents of the /content directory to understand the dataset structure
print(os.listdir('/content/Blood cell Cancer [ALL]'))
```

```
['[Malignant] early Pre-B', '[Malignant] Pre-B', 'Benign', '[Malignant] Pro-B']
```

```
data_dir = '/content/Blood cell Cancer [ALL]'
image_size = (256, 256)
batch_size = 32
train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="training", seed=123,
    image_size=image_size,
    batch_size=batch_size
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=image_size,
    batch_size=batch_size
)
class_names = train_ds.class_names
print(f"Class names: {class_names}")
```

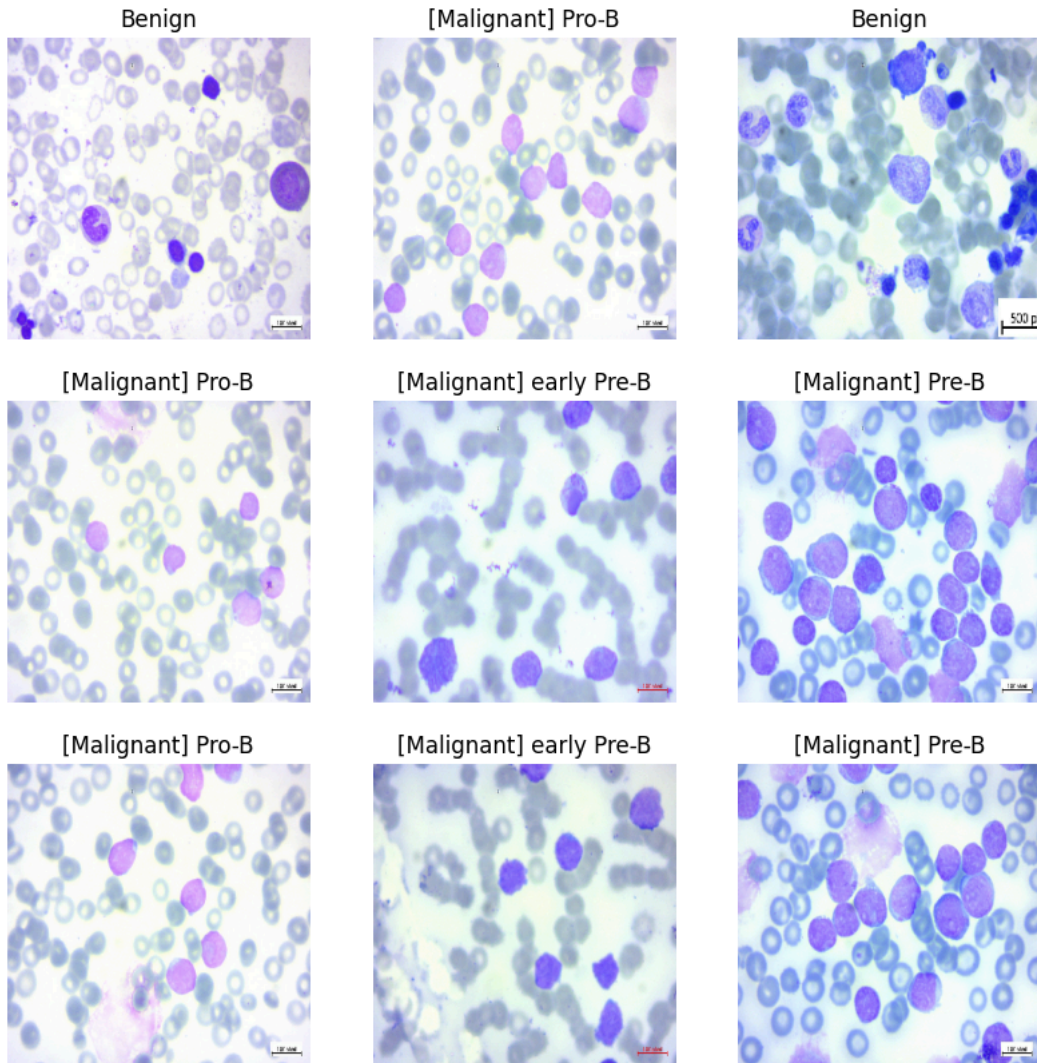
```
Found 3242 files belonging to 4 classes.
Using 2594 files for training.
Found 3242 files belonging to 4 classes.
Using 648 files for validation.
Class names: ['Benign', '[Malignant] Pre-B', '[Malignant] Pro-B', '[Malignant] early Pre-B']
```

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 10))
for images, labels in train_ds.take(1):
    for i in range(9):
```

```

ax = plt.subplot(3, 3, i + 1)
plt.imshow(images[i].numpy().astype("uint8"))
plt.title(class_names[labels[i]])
plt.axis("off")

```



```

AUTOTUNE = tf.data.AUTOTUNE
def preprocess_image(image, label):
    # Normalize pixel values to be between 0 and 1.
    image = tf.cast(image / 255.0, tf.float32)
    return image, label
train_ds = train_ds.map(preprocess_image).cache().prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.map(preprocess_image).cache().prefetch(buffer_size=AUTOTUNE)

```

```

num_classes = len(class_names)
model = keras.Sequential([
    layers.Rescaling(1./255, input_shape=(image_size[0], image_size[1], 3)),
    layers.Conv2D(32, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(128, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dense(num_classes)
])
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=['accuracy']
)
model.summary()

```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/preprocessing/data_layer.py:95: UserWarning: Do not pass an `input`
super().__init__(**kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 256, 256, 3)	0
conv2d (Conv2D)	(None, 254, 254, 32)	896
max_pooling2d (MaxPooling2D)	(None, 127, 127, 32)	0
conv2d_1 (Conv2D)	(None, 125, 125, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 62, 62, 64)	0
conv2d_2 (Conv2D)	(None, 60, 60, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 30, 30, 128)	0
flatten (Flatten)	(None, 115200)	0
dense (Dense)	(None, 256)	29,491,456
dense_1 (Dense)	(None, 4)	1,028

Total params: 29,585,732 (112.86 MB)

Trainable params: 29,585,732 (112.86 MB)

Non trainable params: 0 (0.00 MB)

```
feature_extractor = keras.Model(inputs=model.inputs, outputs=model.layers[-3].output) # Output
# Extract features for training data
train_features = []
train_labels_extracted = []
for images, labels in train_ds:
    features = feature_extractor(images)
    train_features.append(features.numpy())
    train_labels_extracted.append(labels.numpy())
train_features = np.concatenate(train_features, axis=0)
train_labels_extracted = np.concatenate(train_labels_extracted, axis=0)
# Extract features for validation data
val_features = []
val_labels_extracted = []
for images, labels in val_ds:
    features = feature_extractor(images)
    val_features.append(features.numpy())
    val_labels_extracted.append(labels.numpy())
val_features = np.concatenate(val_features, axis=0)
val_labels_extracted = np.concatenate(val_labels_extracted, axis=0)
print(f"Shape of extracted training features: {train_features.shape}")
print(f"Shape of extracted validation features: {val_features.shape}")
```

Shape of extracted training features: (2594, 115200)
Shape of extracted validation features: (648, 115200)

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
# Initialize and train the Random Forest Classifier
print("\n--- Training Random Forest ---")
random_forest_model = RandomForestClassifier(random_state=42)
random_forest_model.fit(train_features, train_labels_extracted)
# Make predictions on the validation set
y_pred_rf = random_forest_model.predict(val_features)
# Evaluate the Random Forest model
accuracy_rf = accuracy_score(val_labels_extracted, y_pred_rf)
report_rf = classification_report(val_labels_extracted, y_pred_rf, target_names=class_names)
print(f"Random Forest Accuracy: {accuracy_rf:.4f}")
print(f"Random Forest Classification Report:\n{report_rf}")
```

```
--- Training Random Forest ---
Random Forest Accuracy: 0.8503
Random Forest Classification Report:
              precision    recall  f1-score   support

   Benign              0.92       0.33       0.49        106
[Malignant] Pre-B       0.98       0.89       0.93        187
[Malignant] Pro-B       0.84       0.99       0.91        153
[Malignant] early Pre-B 0.76       0.98       0.86        202

   accuracy                   0.85        648
  macro avg              0.88       0.80       0.80        648
 weighted avg              0.87       0.85       0.83        648
```

```
from sklearn.tree import DecisionTreeClassifier
```

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
# Initialize and train the Decision Tree Classifier
print("\n--- Training Decision Tree ---")
decision_tree_model = DecisionTreeClassifier(random_state=42)
decision_tree_model.fit(train_features, train_labels_extracted)
# Make predictions on the validation set
y_pred_dt = decision_tree_model.predict(val_features)
# Evaluate the Decision Tree model
accuracy_dt = accuracy_score(val_labels_extracted, y_pred_dt)
report_dt = classification_report(val_labels_extracted, y_pred_dt, target_names=class_names)
print(f"Decision Tree Accuracy: {accuracy_dt:.4f}")
print(f"Decision Tree Classification Report:\n{report_dt}")

```

```

--- Training Decision Tree ---
Decision Tree Accuracy: 0.7145
Decision Tree Classification Report:

```

	precision	recall	f1-score	support
Benign	0.38	0.38	0.38	106
[Malignant] Pre-B	0.75	0.76	0.76	187
[Malignant] Pro-B	0.80	0.82	0.81	153
[Malignant] early Pre-B	0.79	0.76	0.77	202
accuracy			0.71	648
macro avg	0.68	0.68	0.68	648
weighted avg	0.71	0.71	0.71	648

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
# Initialize and train the K-Nearest Neighbors Classifier
print("\n--- Training K-Nearest Neighbors ---")
knn_model = KNeighborsClassifier(n_neighbors=5) # You can adjust n_neighbors
knn_model.fit(train_features, train_labels_extracted)
# Make predictions on the validation set
y_pred_knn = knn_model.predict(val_features)
# Evaluate the K-Nearest Neighbors model
accuracy_knn = accuracy_score(val_labels_extracted, y_pred_knn)
report_knn = classification_report(val_labels_extracted, y_pred_knn, target_names=class_names)
print(f"K-Nearest Neighbors Accuracy: {accuracy_knn:.4f}")
print(f"K-Nearest Neighbors Classification Report:\n{report_knn}")

```

```

--- Training K-Nearest Neighbors ---
K-Nearest Neighbors Accuracy: 0.4259
K-Nearest Neighbors Classification Report:

```

	precision	recall	f1-score	support
Benign	0.43	0.36	0.39	106
[Malignant] Pre-B	1.00	0.30	0.46	187
[Malignant] Pro-B	0.33	0.99	0.49	153
[Malignant] early Pre-B	0.73	0.15	0.25	202
accuracy			0.43	648
macro avg	0.62	0.45	0.40	648
weighted avg	0.66	0.43	0.39	648